

Week 2 Assignment: REINFORCE on CartPole

Overview

In this assignment, you will implement and train your first policy gradient agent using the REINFORCE algorithm on CartPole-v1. By the end, you should have a working training script, basic plots, and written reflections.

Setup

Before starting, make sure your environment from Week 1 still works.

Environment Check

Create a file `week2_check_env.py` with:

```
import gymnasium as gym

env = gym.make("CartPole-v1", render_mode=None)
obs, info = env.reset()
print("Initial observation:", obs)
print("Action space:", env.action_space)
env.close()
```

Run it with:

```
python week2_check_env.py
```

If it runs without errors, you are ready.

Project Structure

By the end of Week 2, your folder might look like:

```
project_root/
  week1_explore.py
  week1_notes.md
  week2_reinforce.py
  week2_notes.md
  plots/
    cartpole_rewards.png
```

Task 1: Implement REINFORCE (Core)

Create `week2_reinforce.py` and start from this skeleton:

```
# week2_reinforce.py

import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

class PolicyNetwork(nn.Module):
    def __init__(self, state_dim, hidden_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, action_dim),
        )

    def forward(self, state):
        """state: tensor (batch_size, state_dim) -> probs (batch_size,
            action_dim)"""
        logits = self.net(state)
        probs = torch.softmax(logits, dim=-1)
        return probs

def run_episode(env, policy, render=False, gamma=0.99):
    states, actions, rewards, log_probs = [], [], [], []

    obs, info = env.reset()
    done = False

    while not done:
        state = torch.from_numpy(obs).float().unsqueeze(0)
        with torch.no_grad():
            probs = policy(state)
        dist = torch.distributions.Categorical(probs=probs)
        action = dist.sample()

        if render:
            env.render()

        next_obs, reward, terminated, truncated, info = env.step(action.item())
        done = terminated or truncated

        states.append(obs)
        actions.append(action.item())
        rewards.append(reward)
        log_probs.append(dist.log_prob(action))

        obs = next_obs
```

```

returns = []
G = 0.0
for r in reversed(rewards):
    G = r + gamma * G
    returns.insert(0, G)

returns = torch.tensor(returns, dtype=torch.float32)
log_probs = torch.stack(log_probs)

return states, actions, rewards, returns, log_probs

def train_reinforce(
    env_name="CartPole-v1",
    hidden_dim=64,
    learning_rate=1e-2,
    gamma=0.99,
    num_episodes=500,
):
    env = gym.make(env_name, render_mode=None)
    state_dim = env.observation_space.shape[0]
    action_dim = env.action_space.n

    policy = PolicyNetwork(state_dim, hidden_dim, action_dim)
    optimizer = optim.Adam(policy.parameters(), lr=learning_rate)

    all_episode_returns = []

    for episode in range(num_episodes):
        states, actions, rewards, returns, log_probs = run_episode(
            env, policy, render=False, gamma=gamma
        )

        returns_mean = returns.mean()
        returns_std = returns.std() + 1e-8
        normalized_returns = (returns - returns_mean) / returns_std

        loss = -(log_probs * normalized_returns).sum()

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        episode_return = sum(rewards)
        all_episode_returns.append(episode_return)

    if (episode + 1) % 10 == 0:
        avg_last_10 = np.mean(all_episode_returns[-10:])
        print(
            f"Episode {episode + 1:4d} | "
            f"Return: {episode_return:6.1f} | "
            f"Avg last 10: {avg_last_10:6.1f}"
        )

```

```

env.close()
return all_episode_returns, policy

if __name__ == "__main__":
    returns, trained_policy = train_reinforce()

```

What You Must Do

1. Read through the skeleton and add comments in your own words explaining:
 - How actions are sampled from the policy.
 - How returns are computed.
 - How the REINFORCE loss is formed and why it has a minus sign.
2. Run the script for at least 300–500 episodes and confirm returns generally improve.

Task 2: Inspect Learning and Plot Returns

After training, inspect how learning progresses.

Logging

Make sure you keep track of all episode returns in a list `all_episode_returns` as in the skeleton.

Optional: Plotting

Add a small plotting helper (e.g., in the same file or a separate one):

```

import matplotlib.pyplot as plt

def plot_returns(returns, window=10, filename="plots/cartpole_rewards.png"):
    import os
    os.makedirs("plots", exist_ok=True)

    returns = np.array(returns)
    x = np.arange(len(returns))

    if len(returns) >= window:
        sma = np.convolve(returns, np.ones(window)/window, mode="valid")
    else:
        sma = returns

    plt.figure(figsize=(8, 4))
    plt.plot(x, returns, alpha=0.3, label="Episode return")
    plt.plot(np.arange(len(sma)), sma, label=f"{window}-episode moving avg")
    plt.xlabel("Episode")
    plt.ylabel("Return")
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename)

```

```
plt.close()
```

Call it after training to save `plots/cartpole_rewards.png`.

Task 3: Written Reflection (`week2_notes.md`)

Create `week2_notes.md` and answer in 2–4 sentences each:

1. Describe the trend in episode returns. Does the moving average increase? Does it plateau or oscillate?
2. Compare the trained policy to a random policy when you run a few episodes with rendering. What qualitative differences do you see?
3. Try at least one hyperparameter change (learning rate, number of episodes, hidden size, or discount factor). What changed in the learning behavior?
4. In your own words, what does the REINFORCE loss $-\log \pi(a_t | s_t)G_t$ try to do?

If you attempt any optional experiments (e.g., different γ , removing normalization), briefly summarize what you observed.

Deliverables

By the end of Week 2, you should have:

- **Code:** `week2_reinforce.py` (and optionally `week2_check_env.py`).
- **Notes:** `week2_notes.md` with your written answers.
- **Plot (optional but recommended):** `plots/cartpole_rewards.png`.

During your mentor meeting, walk through:

- Your reward curve and what you think it shows.
- Any difficulties you faced implementing or stabilizing REINFORCE.
- How this simple policy gradient might extend to more complex tasks later in the project.